

# Laplace Solver on HPC

CS-435: Parallel & Distributed Processing

|                  |                                            |
|------------------|--------------------------------------------|
| Instructor       | Attique Dawood                             |
| Course           | CS-435 — Parallel & Distributed Processing |
| Due Date         | 18 May 2026                                |
| Head Node        | afrit                                      |
| Compiler / Build | GCC 4.4.7 / CMake 2.8.12 / Open MPI 1.6.2  |

## Introduction

The Laplace equation ( $\nabla^2 u = 0$ ) describes steady-state heat distribution, electrostatic potential, and fluid flow across a 2D domain. Solving it numerically via iterative Jacobi relaxation is computationally intensive for large grids, making it an ideal candidate for parallelisation. This project implements the solver in three progressive stages: Serial, OpenMP (shared-memory), and MPI (distributed-memory), deployed on the afrit HPC cluster.

The objectives are: (1) characterise the cluster's hardware and software environment, (2) implement and verify correctness of all three solver variants, and (3) empirically analyse scaling behaviour through wall-time benchmarks across increasing domain sizes.

|                             |                               |                                     |                               |
|-----------------------------|-------------------------------|-------------------------------------|-------------------------------|
| <b>3</b><br>Implementations | <b>4</b><br>Grid Sizes Tested | <b>2.51x</b><br>Peak OpenMP Speedup | <b>0</b><br>Correctness Error |
|-----------------------------|-------------------------------|-------------------------------------|-------------------------------|

## PART I — Getting to Know the HPC Cluster

### 1. Cluster Map & Node Availability

The cluster consists of one head node (afrit) and 32 compute nodes (compute-0-0 through compute-0-31). The SGE job scheduler is non-functional — all qstat and qhost commands return "critical error: can't resolve group" — so each node was assessed individually via manual SSH and inspection of uptime and top output.

#### 1.1 Head Node

| Node  | Status     | Load Avg | Cores | Notes                                   |
|-------|------------|----------|-------|-----------------------------------------|
| afrit | OVERLOADED | 48.12    | 16    | HEAD NODE — Do not run simulations here |

#### 1.2 Compute Nodes

Colour coding: Green = Idle, Red = Offline, Orange = Locked, Teal = Active/Busy

| IDLE             |                 | OFFLINE        |                | LOCKED          |                 | ACTIVE          |                 |
|------------------|-----------------|----------------|----------------|-----------------|-----------------|-----------------|-----------------|
| 0-0<br>IDLE      | 0-1<br>OFFLINE  | 0-2<br>OFFLINE | 0-3<br>OFFLINE | 0-4<br>OFFLINE  | 0-5<br>IDLE     | 0-6<br>IDLE     | 0-7<br>OFFLINE  |
| 0-8<br>ACTIVE    | 0-9<br>IDLE     | 0-10<br>IDLE   | 0-11<br>IDLE   | 0-12<br>IDLE    | 0-13<br>IDLE    | 0-14<br>OFFLINE | 0-15<br>OFFLINE |
| 0-16<br>IDLE     | 0-17<br>IDLE    | 0-18<br>IDLE   | 0-19<br>IDLE   | 0-20<br>OFFLINE | 0-21<br>OFFLINE | 0-22<br>OFFLINE | 0-23<br>LOCKED  |
| 0-24<br>IDLE     | 0-25<br>OFFLINE | 0-26<br>IDLE   | 0-27<br>IDLE   | 0-28<br>OFFLINE | 0-29<br>OFFLINE | 0-30<br>OFFLINE | 0-31<br>LOCKED  |
| 15 Online / Idle |                 | 14 Offline     |                | 2 Locked        |                 | 1 Active        |                 |

#### ⚙ Job Scheduler Broken

SGE commands qstat, qhost, and qsub all fail with "critical error: can't resolve group." Node status was assessed by manual SSH. This prevented proper job dispatching to compute nodes for Part II benchmarks.

### 2. Hardware Specifications

| [STANDARD] Standard Node — afrit & most compute nodes |                                      |
|-------------------------------------------------------|--------------------------------------|
| CPU Model                                             | Intel Xeon E5520 @ 2.27 GHz          |
| Logical CPUs                                          | 16 (2 Sockets × 4 Cores × 2 Threads) |
| NUMA Nodes                                            | 2                                    |
| Cache L1/L2/L3                                        | 32 KB / 256 KB / 8 MB                |
| RAM                                                   | ~24 GB                               |

|     |      |
|-----|------|
| GPU | None |
|-----|------|

| [HIGH-MEM] High-Memory Super Node — compute-0-8 |                                                      |
|-------------------------------------------------|------------------------------------------------------|
| CPU Model                                       | GenuineIntel Model 44 @ 1.60 GHz (Xeon X5600 series) |
| Logical CPUs                                    | 24 (2 Sockets × 6 Cores × 2 Threads)                 |
| NUMA Nodes                                      | 2                                                    |
| Cache L1/L2/L3                                  | 32 KB / 256 KB / 12 MB                               |
| RAM                                             | ~48 GB (most powerful node in cluster)               |
| GPU                                             | None                                                 |

## 3. Network Interconnects & Software Environment

### 3.1 Network

|              |                                                                    |
|--------------|--------------------------------------------------------------------|
| Interconnect | Mellanox InfiniBand MT26428 — 40 Gbps QDR                          |
| Protocol     | Low-latency RDMA-capable interconnect linking all compute nodes    |
| MPI Module   | rocks-openmpi_ib — InfiniBand-optimised MPI (critical for Part II) |

### 3.2 Software Environment

| Package          | Version  | Purpose                         |
|------------------|----------|---------------------------------|
| GCC              | 4.4.7    | C/C++ compiler                  |
| Open MPI         | 1.6.2    | Message passing interface       |
| CMake            | 2.8.12.2 | Build system                    |
| rocks-openmpi_ib | —        | InfiniBand-optimised MPI module |

#### Critical: Storage at 100% Capacity

Partition /state/partition1/home3 (all user home directories) is at 3.8 TB — 100% full. Output files from simulations may fail silently. All output directories must be verified before running Part II jobs.

## 4. Busy Nodes & Rule Violations

#### Severe Violation — Head Node Abuse Detected

User suneela.arif is actively running multiple `mpirun -np 32` and `mpirun -np 16` processes directly on afrit, driving the load average to 48.12 on a 16-core system. This directly violates Note 3 of the project specification: "Avoid running simulations on the head node afrit." The resulting resource starvation degrades the environment for all users and skews benchmark timings.

### Observed Load Averages

| Node                   | Load Average | Interpretation                             |
|------------------------|--------------|--------------------------------------------|
| afrit (head node)      | 48.12        | 300% overloaded (16-core system)           |
| compute-0-8            | ~2.00        | Active, only ~8% utilised (24-core system) |
| All other online nodes | ~0.00        | Idle                                       |

## 5. Node Rankings

### 5.1 By Performance

| Rank | Node                    | Cores | RAM   | L3 Cache | Notes          |
|------|-------------------------|-------|-------|----------|----------------|
| #1   | compute-0-8             | 24    | 48 GB | 12 MB    | Super node     |
| #2   | All other compute nodes | 16    | 24 GB | 8 MB     | Standard nodes |
| —    | afrit (head node)       | 16    | 24 GB | 8 MB     | DO NOT USE     |

### 5.2 By Availability

| Tier | Nodes                                              | Status     | Recommendation                         |
|------|----------------------------------------------------|------------|----------------------------------------|
| 1    | 0-6, 0-9, 0-11, 0-12, 0-13, 0-16, 0-17, 0-24, 0-27 | Fully Idle | Best for MPI jobs — maximum free RAM   |
| 2    | 0-0, 0-5, 0-10, 0-18, 0-19, 0-26                   | Idle       | Good — some cached RAM present         |
| 3    | compute-0-8                                        | Low Load   | Good for large-memory jobs (48 GB RAM) |
| 4    | 0-1..4, 0-7, 0-14, 0-15, 0-20..22, 0-25, 0-28..30  | Offline    | Not available                          |
| 5    | 0-23, 0-31                                         | Locked     | Require password credentials           |
| X    | afrit                                              | AVOID      | Strictly prohibited — project Note 3   |

## PART II — Laplace Solver: Implementation & Analysis

### 6. Implementation Approach

The 2D Laplace equation is solved by iterative Jacobi relaxation: at each epoch every interior cell is updated to the average of its four neighbours. Convergence is measured by the maximum change across all cells. Three implementations were developed, each building on the last.

#### 6.1 Serial Baseline

The serial implementation uses a 1D flattened vector<double> to store the  $N \times N$  grid in row-major order. Each epoch iterates over all interior cells  $i \times j$  and computes the four-neighbour average. Wall time is measured using `gettimeofday()` from `sys/time.h` (required by GCC 4.4.7, which lacks `std::chrono`).

#### 6.2 OpenMP Implementation

- Inner loop over grid rows parallelised with `#pragma omp parallel for`
- Reduction clause used to compute `max_diff` safely across threads
- No explicit synchronisation needed — threads share the same address space
- Thread count configured via `OMP_NUM_THREADS` environment variable
- Grid stored as 1D flattened vector<double> for cache-line efficiency

#### 6.3 MPI Implementation (Domain Decomposition + Halo Exchange)

The domain is decomposed into horizontal row-slabs — one slab per MPI process. Each process owns  $N/P$  interior rows plus two ghost (halo) rows for its boundaries.

- Each epoch: `MPI_Sendrecv` exchanges top/bottom boundary rows with neighbouring processes
- Global convergence checked via `MPI_Allreduce(MPI_MAX)` across all ranks
- Root process (rank 0) gathers all slabs via `MPI_Gather` and writes the final grid
- Correctness verified: `max_diff` between serial and MPI output = 0.0

#### CMake Packaging

The project is packaged with CMake so an end-user can build all targets with: `cmake .. && make`. Three targets are produced: `laplace_serial`, `laplace_omp`, and `laplace_mpi`. The `CMakeLists.txt` uses legacy `find_package` syntax compatible with CMake 2.8.

### 7. Problems Faced & Solutions

The legacy environment on afrit (GCC 4.4.7, CMake 2.8) predates modern C++ by several years. Three categories of incompatibility were encountered and resolved.

#### Problem 1 — CMake Toolchain Obsolescence

##### Problem

CMake 2.8 does not support `target_compile_features(cxx_std_17)` or modern `find_package` idioms.

**Fix**

Rewrote CMakeLists.txt using legacy find\_package syntax, manual include\_directories(\${MPI\_INCLUDE\_PATH}), and explicit target\_link\_libraries(). Compiler flags appended directly via set(CMAKE\_CXX\_FLAGS ...).

**Problem 2 — C++11 / C++17 Incompatibility**

| Issue            | Modern Code                        | GCC 4.4.7 Fix                        |
|------------------|------------------------------------|--------------------------------------|
| Type aliases     | using Grid = vector<double>        | typedef vector<double> Grid          |
| String to int    | std::stoi(argv[1])                 | atoi(argv[1])                        |
| Timing           | std::chrono::high_resolution_clock | gettimeofday() from sys/time.h       |
| Template spacing | vector<vector<double>>             | vector<vector<double> > (note space) |

**Problem 3 — Broken SGE Job Scheduler****⚠ Broken qsub — Scheduler Error**

The qsub job submission system fails with "can't resolve group", making it impossible to dispatch jobs to dedicated compute nodes. All benchmarks were therefore collected on the head node afrit, which was simultaneously overloaded (load avg 48.12). This means MPI timings are artificially inflated and do not represent true inter-node InfiniBand performance.

**8. Experimental Results & Analysis****8.1 OpenMP — Serial vs Parallel Performance**

| Grid (N×N)         | Serial (s) | OpenMP (s) | Speedup | Status     |
|--------------------|------------|------------|---------|------------|
| <b>128 × 128</b>   | 0.022      | 0.013      | 1.73x   | Scaling up |
| <b>256 × 256</b>   | 0.151      | 0.060      | 2.51x   | PEAK       |
| <b>512 × 512</b>   | 1.118      | 0.444      | 2.51x   | PEAK       |
| <b>1024 × 1024</b> | 15.372     | 9.377      | 1.63x   | Degraded   |

**📊 Analysis — Why Speedup Drops at N=1024**

The 2.51x plateau at N=256 and N=512 reflects Amdahl's Law — approximately 40% of execution remains serial (boundary handling, convergence check, memory allocation). At N=1024, each row contains 1024 doubles (8 KB). The full 1024x1024 grid occupies ~8 MB, which EXCEEDS the 8 MB L3 cache of standard nodes. Repeated cache misses on every epoch dramatically increase memory access latency, shrinking the parallelism benefit. At large N, memory bandwidth — not computation — becomes the bottleneck, and multiple threads compete for the same memory bus.

**8.2 MPI Performance Results**

| Configuration       | Grid N | Epochs | Processes | Wall Time (s) |
|---------------------|--------|--------|-----------|---------------|
| <b>Small scale</b>  | 64     | 50     | 2         | 0.0007        |
| <b>Medium scale</b> | 1000   | 1000   | 4         | 28.2          |

**⚠ MPI Results — Important Caveats**

MPI benchmarks were run on the head node due to the broken qsub scheduler. On a single machine, MPI processes communicate via shared memory rather than InfiniBand. More critically, afrit was running at load average 48.12 during testing, introducing severe OS scheduling noise. True MPI performance across dedicated compute nodes via the 40 Gbps InfiniBand fabric would be substantially better and is expected to achieve near-linear speedup for large grids where computation dominates communication.

**✓ Correctness Verified**

For all grid sizes and both parallel implementations (OpenMP and MPI), the maximum absolute difference between the parallel output and the serial reference was 0.0. The halo exchange boundary conditions and OpenMP reduction clauses are implemented correctly.

## 9. Conclusion

This project demonstrated that the 2D Laplace solver scales well with both OpenMP and MPI parallelism for medium grid sizes, achieving a peak speedup of 2.51x with OpenMP on 16 cores. Performance degrades for very large grids due to L3 cache exhaustion and memory-bandwidth saturation — a hardware limitation, not an algorithmic flaw.

The cluster investigation revealed a challenging environment: a broken job scheduler, a storage partition at 100% capacity, and a severely overloaded head node. These factors prevented true multi-node MPI benchmarking via InfiniBand, which would be the natural next step. Under ideal conditions — dedicated compute nodes, InfiniBand transport via `rocks-openmpi_ib`, and an operational `qsub` scheduler — MPI domain decomposition is expected to scale near-linearly for large  $N$ .

### Key Takeaways

|                                                                                                                                                                            |                                                                                                                                                                                             |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Parallelism delivers real gains</b><br>Both OpenMP and MPI produce correct results and deliver meaningful speedups for medium-sized domains ( $N=256$ to $N=512$ ).     | <b>Memory bandwidth is the ceiling</b><br>At $N=1024$ the grid exceeds L3 cache. Future work should explore cache-blocking or tiling strategies.                                            |
| <b>Infrastructure matters</b><br>A broken scheduler, full storage, and shared-node noise all degrade measured performance. Cluster health is as critical as the algorithm. | <b>InfiniBand potential untapped</b><br>The 40 Gbps InfiniBand fabric was not exercised. Dedicated compute nodes with <code>rocks-openmpi_ib</code> should unlock true distributed speedup. |